

2010376103 CSE4261 Assignment-4

Omor Niloy

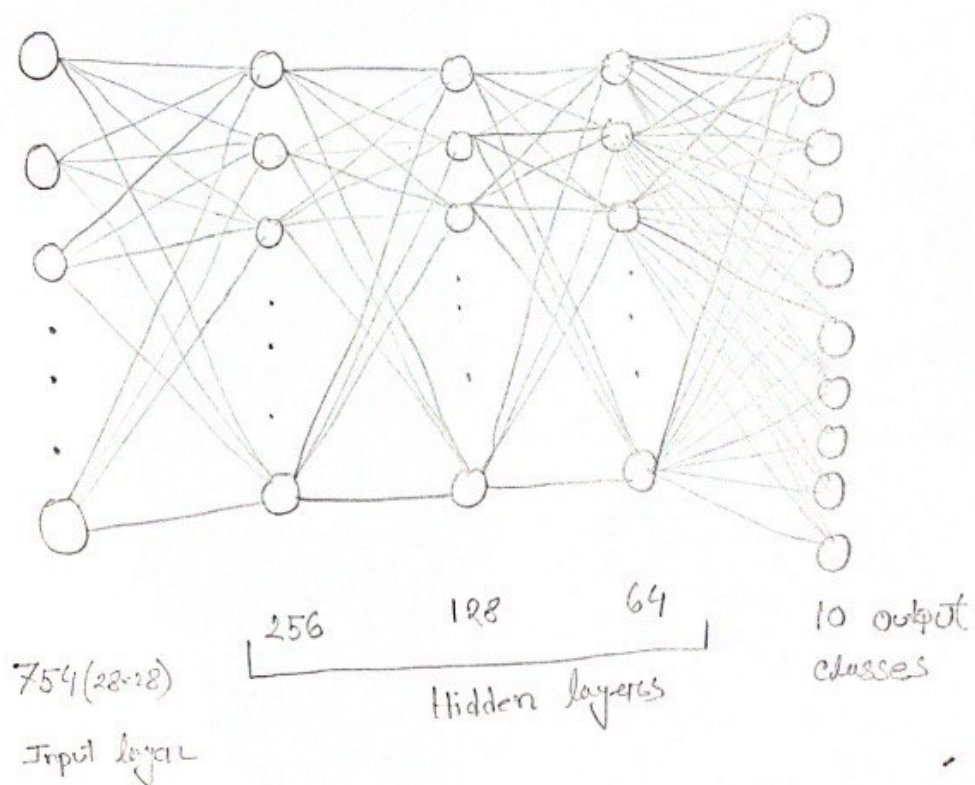
June 2025

Assignment - 4

Omara Faruk Nilay

ID: 2010376103

1. A neural network having 3 hidden layers for classifying 10 classes:



Forward propagation computation graph:

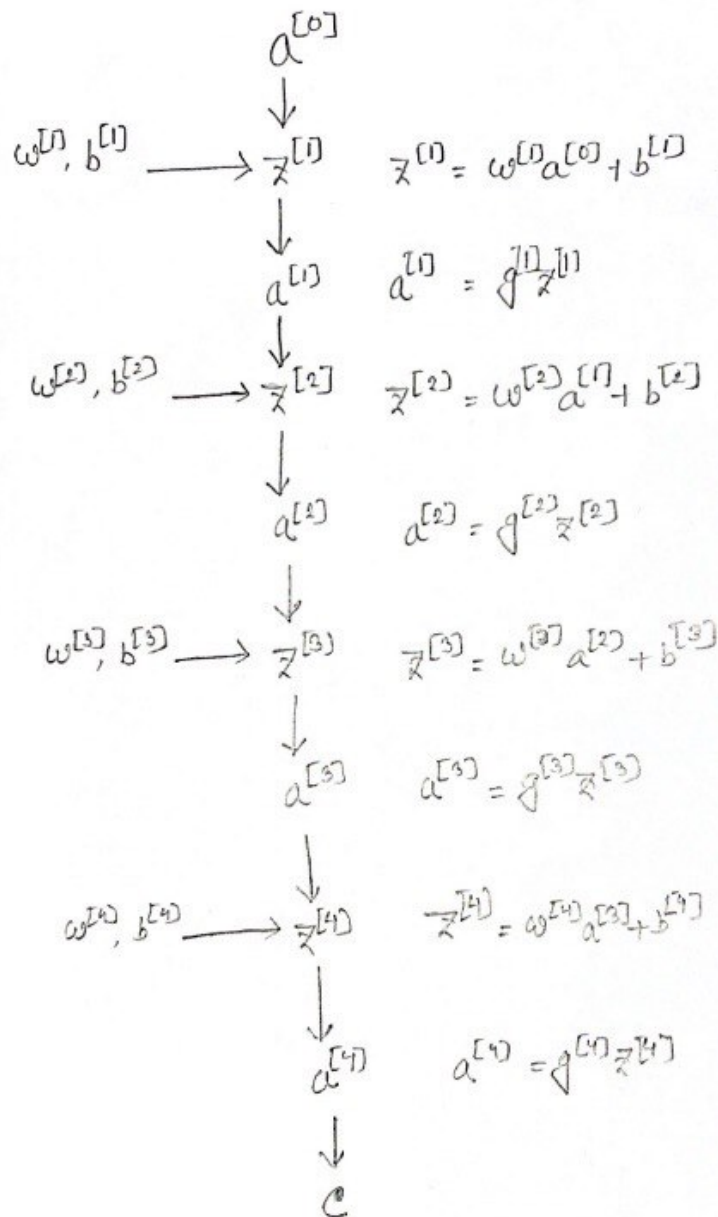


Fig: Forward propagation computation graph

Back propagation computation graph

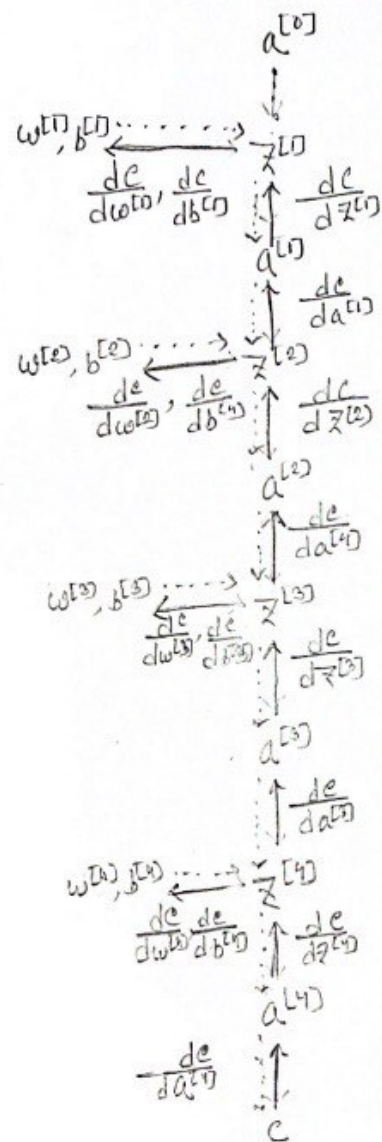


Fig: Back propagation computation graph

Equations of updating weights:

for layer: 4,

$$\frac{dc}{dw^{(4)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{dw^{(4)}}$$

$$\frac{dc}{db^{(4)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{db^{(4)}}$$

for layer: 3,

$$\frac{dc}{dw^{(3)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{dw^{(3)}}$$

$$\frac{dc}{db^{(3)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{db^{(3)}}$$

for layer: 2,

$$\frac{dc}{dw^{(2)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{dw^{(2)}}$$

$$\frac{dc}{db^{(2)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{db^{(2)}}$$

for layer: 1,

$$\frac{dc}{dw^{(1)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{da^{(1)}} \cdot \frac{da^{(1)}}{dz^{(1)}} \cdot \frac{dz^{(1)}}{dw^{(1)}}$$

$$\frac{dc}{db^{(1)}} = \frac{dc}{da^{(4)}} \cdot \frac{da^{(4)}}{dz^{(4)}} \cdot \frac{dz^{(4)}}{da^{(3)}} \cdot \frac{da^{(3)}}{dz^{(3)}} \cdot \frac{dz^{(3)}}{da^{(2)}} \cdot \frac{da^{(2)}}{dz^{(2)}} \cdot \frac{dz^{(2)}}{da^{(1)}} \cdot \frac{da^{(1)}}{dz^{(1)}} \cdot \frac{dz^{(1)}}{db^{(1)}}$$

1 Dataset

The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. For this experiment:

- 60,000 images used for training/validation (split 90% train, 10% validation)
- 10,000 images used for testing
- Pixel values normalized to $[0,1]$
- Labels converted to one-hot encoding (10 classes)

The dataset was also converted to 4D tensors with shape (batch, 28, 28, 1) to feed into the neural network.

2 Experimental Setup

A fully-connected feedforward neural network was used:

- Input: Flattened 28×28 images
- Hidden layers: $256 \rightarrow 128 \rightarrow 64$ neurons with ReLU activation
- Output layer: 10 logits (no softmax)

Two training approaches were compared:

2.1 Manual Training with `tf.GradientTape()`

- Loss: Categorical cross-entropy
- Optimizer: Adam
- Training loop: Gradient computed via `tf.GradientTape()` and applied manually
- Metrics: Train/validation loss and accuracy computed after each epoch
- Epochs: 5, Batch size: 64

2.2 Standard Training with `model.fit()`

- Loss: Categorical cross-entropy (from logits)
- Optimizer: Adam
- Training handled by `model.fit()`
- Validation data used for monitoring
- Epochs: 5, Batch size: 64

Finally, test accuracy was evaluated for both approaches using a separate test set of 10,000 MNIST images.

3 Results

Observations:

- Both training methods achieve very similar test accuracies on MNIST.
- Manual training with `tf.GradientTape()` slightly outperforms `model.fit()` in this run.
- Manual loops allow full control over gradient computation and custom metrics.
- `model.fit()` is simpler and easier for standard training tasks.
- Overall, both approaches are effective for MNIST classification with a fully-connected network.

Table 1: Comparison of Manual `tf.GradientTape()` vs. `model.fit()` Training

Training Method	Test Accuracy (%)
Manual <code>tf.GradientTape()</code>	97.69
<code>model.fit()</code>	97.51

4 Code

[Github code link](#)